

Explain Remaining Balance Logic

The key idea is: this is a DFS + memoization DP solution where `balance` represents how many unmatched `(` we currently have. For every path from `(0,0)` to `(m-1,n-1)`, we are effectively building a parentheses string.

1. What is the problem actually asking?

Suppose the grid is:

```
text
( ( )
( ) )
```

You can move only:

- ↓ down
- → right

For example:

```
text
( → (
↓ ↓
( → )
```

Every visited cell contributes either:

```
text
'(' → balance + 1
')' → balance - 1
```

A path is valid if the resulting parentheses string is valid.

For a valid parentheses string:

1. Balance can**never** become **negative**.
2. At the destination, balance must be**exactly 0**.

Example:

```
text
( ( ) )
+1 +1 -1 -1
1 2 1 0 ✓
```

But:

```
text
) ( ) (
-1 ...
```

Immediately invalid because balance becomes `-1`.

2. What does `solve(i, j, balance)` mean?

This is the most important part.

```
C++
bool solve(int i, int j, int balance, vector<vector<char>>& grid)
```

means:

"If I am currently at cell `(i,j)` and I currently have `balance` unmatched opening brackets, can I reach the destination and form a valid parentheses sequence?"

So our DP state is:

```
text
(i, j, balance)
```

There are three things that determine the future:

State	Meaning
<code>i</code>	current row
<code>j</code>	current column
<code>balance</code>	unmatched <code>(</code> count

3. Why do we need `balance`?

Consider:

```
text
( ( )
```

After these characters:

```
text
(
(
)
```

the balance is:

```
text
1
2
1
```

There is still one unmatched `(`.

So if we reach some cell with:

```
text
balance = 1
```

we need at least one future `)` to close it.

That's why two paths reaching the **same cell** aren't necessarily equivalent.

For example:

```
text
solve(2, 2, 0)
```

and

```
text
solve(2, 2, 2)
```

are completely different states.

The first one needs no closing brackets.

The second one needs at least two `)` somewhere later.

Hence:

```
C++
dp[i][j][balance]
```

4. First check: negative balance

```
C++
if (balance < 0)
    return false;
```

A valid parentheses sequence can never have more closing brackets than opening brackets at any prefix.

For example:

```
text
)
```

gives:

```
text
balance = -1
```

Invalid immediately.

So:

```
text
balance < 0
  ↓
false
```

5. Boundary check

```
C++  
if (i >= m || j >= n)  
    return false;
```

If we move outside the grid, that path is invalid.

For example, from the last row:

```
text  
↓
```

takes us outside.

Similarly, from the last column:

```
text  
→
```

takes us outside.

6. Apply the current cell

This is where the current character affects the balance.

```
C++  
if (grid[i][j] == '(')  
    balance++;  
else  
    balance--;
```

For:

```
text  
(
```

we do:

```
text  
balance++
```

For:

```
text  
)
```

we do:

```
text  
balance--
```

Notice something important:

You call:

```
C++  
solve(0, 0, 0, grid)
```

with balance 0.

But the starting cell hasn't been processed yet.

So inside `solve`, the first cell is processed:

```
text  
grid[0][0] = '('  
  
balance = 0  
    ↓  
balance = 1
```

That's correct.

7. Check balance again

```
C++  
if (balance < 0)  
    return false;
```

This check is actually important because the current cell might have been:

```
text
')'
```

Example:

```
text
balance = 0

current cell = ')'

balance--
    ↓
balance = -1
```

Therefore:

```
C++
return false;
```

8. Now the important part: remaining

You asked specifically about this:

```
C++
int remaining = (m - 1 - i) + (n - 1 - j);
```

This calculates:

How many cells/steps are still left before reaching the destination.

Let's understand why.

Suppose the grid is:

```
text
0 1 2 3
1
2
3
```

Current position:

```
text
(i, j) = (1, 1)
```

Destination:

```
text
(3, 3)
```

To reach (3,3):

```
text
rows remaining = 3 - 1 = 2
columns remaining = 3 - 1 = 2
```

Therefore:

```
text
remaining = 2 + 2
           = 4
```

You need:

```
text
↓ ↓ → →
```

That's exactly 4 moves.

9. Why does remaining help?

Suppose:

```
text
balance = 5
```

And there are only:

```
text
remaining = 3
```

cells/moves available.

Can we possibly reach balance 0?

No.

Because the **best possible scenario** is that all remaining characters are:

```
text
) ) )
```

So balance becomes:

```
text
5
4
3
2
```

We still have:

```
text
balance = 2
```

at the destination.

Therefore, this path is impossible.

That's what this check does:

```
C++
if (balance > remaining)
    return false;
```

It means:

```
"I have more unmatched ( than the number of cells left that could potentially close them."
```

So there is no possible solution from here.

10. Important subtlety about remaining

Look at:

```
C++
int remaining = (m - 1 - i) + (n - 1 - j);
```

It counts **future moves**, not the current cell.

Why?

Because the current cell has already been processed:

```
C++
if (grid[i][j] == '(')
    balance++;
else
    balance--;
```

After processing the current cell, only the future cells can change the balance.

Therefore:

```
text
remaining
=
number of future cells
```

because every future move enters exactly one new cell.

11. Example of remaining

Suppose:

```
text
grid = 3 x 3
```

and we're at:

```
text
(i,j) = (1,1)
```

Destination:

```
text
(2,2)
```

Then:

```
C++
remaining = (2 - 1) + (2 - 1)
           = 1 + 1
           = 2;
```

There are exactly two future cells:

```
text
→
↓
```

So if:

```
text
balance = 3
remaining = 2
```

we immediately know:

```
text
3 > 2
```

Therefore:

```
C++
return false;
```

There aren't enough future positions to put three closing brackets.

12. Destination condition

```
C++
if (i == m - 1 && j == n - 1)
    return balance == 0;
```

Once we reach the bottom-right cell, we're done.

Now we ask:

```
text
Is balance == 0?
```

If yes:

```
text
valid parentheses
```

If no:

```
text
invalid parentheses
```

Example:

```
text
( ( ) )
```

At destination:

```
text
balance = 0
```

Therefore:

```
C++
return true;
```

But:

```
text
( ( ) (
```

might finish with:

```
text
balance = 2
```

So:

```
C++
return false;
```

13. Memoization

Now:

```
C++
if (dp[i][j][balance] != -1)
    return dp[i][j][balance];
```

This is standard memoization.

Suppose we have already calculated:

```
text
solve(2,3,1)
```

and discovered:

```
text
false
```

Later, another path reaches:

```
text
(2,3)
```

with:

```
text
balance = 1
```

There is no reason to calculate it again.

We simply return:

```
C++
dp[2][3][1]
```

14. The actual movement

From every cell, we have two choices:

```
C++
bool down = solve(i + 1, j, balance, grid);
bool right = solve(i, j + 1, balance, grid);
```

Visually:

```
text
      current
      (i,j)
     /   \
    ↓     →
  (i+1,j) (i,j+1)
```

We need **at least one** valid path.

Therefore:

```
C++
down || right
```

If either direction eventually produces a valid parentheses sequence:

text

true

15. Complete flow of solve()

Think of the function in this exact order:

text

`solve(i, j, balance)`



Is balance negative?



YES → false



NO



Outside grid?



YES → false



NO



Process current cell

'(' → balance++

')' → balance--



Balance negative?



YES → false



NO



Can remaining cells close balance?



balance > remaining



YES → false



NO



Destination?



YES → balance == 0



NO



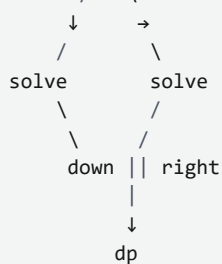
Already calculated?



YES → return dp



NO



16. Let's do a small dry run

Consider:

text

((

))

Grid:

```
text
( (
) )
```

Coordinates:

```
text
(0,0) (0,1)
(1,0) (1,1)
```

Start:

```
C++
solve(0,0,0)
```

Current:

```
text
(
```

So:

```
text
balance = 1
```

Remaining:

```
C++
(1 - 0) + (1 - 0)
= 2
```

So:

```
text
balance = 1
remaining = 2
```

Possible.

Now go right:

```
C++
solve(0,1,1)
```

Current:

```
text
(
```

Balance:

```
text
2
```

Remaining:

```
text
1
```

Now:

```
C++
balance > remaining
2 > 1
```

Therefore:

```
C++
return false;
```

That branch can never close both opening brackets.

Instead go down:

```
C++
solve(1,0,1)
```

Current:

text

)

Balance:

text

0

Remaining:

text

1

Go right:

C++

```
solve(1,1,0)
```

Current:

text

)

Balance:

text

-1

Therefore:

C++

```
return false;
```

So this particular path doesn't work either.

Actually, notice that the grid

text

```
( (
) )
```

has two possible paths:

text

```
( ( ) )
```

and

text

```
( ) ( )
```

The second path is valid:

text

```
( → (
↓
)
→ )
```

Let's trace that path:

text

```
(
balance = 1

)
balance = 0

(
balance = 1

)
balance = 0
```

Destination:

```
text
balance == 0
```

Therefore:

```
text
true
```

17. Why remaining is an optimization, not the core DP

This distinction is important for understanding the code.

The actual DP idea is:

```
text
(i, j, balance)
```

The remaining condition:

```
C++
if (balance > remaining)
    return false;
```

is a pruning optimization.

Without it, the algorithm can still be correct.

But it may explore many useless states.

With it, we eliminate states where success is mathematically impossible.

Think of it as:

```
"I need 5 closing brackets, but I only have 3 positions left. Stop exploring this branch."
```

18. One more important observation

There is actually another mathematical condition that can be used here:

```
C++
if (balance > remaining)
    return false;
```

is valid because every future cell can decrease balance by **at most** 1.

But there is also a parity consideration.

For example:

```
text
balance = 2
remaining = 1
```

Obviously impossible.

Even when:

```
text
balance <= remaining
```

the exact number of `)` needed must fit the remaining positions.

However, because the path length is fixed and the final balance must be zero, the parity naturally aligns with the total path length condition:

```
C++
int len = m + n - 1;

if (len % 2 == 1)
    return false;
```

A valid parentheses sequence must have an even number of characters.

19. Why len % 2 == 1?

A valid parentheses string always has equal:

```
text
'(' count = ')' count
```

Therefore its length must be even.

The path always visits:

```
text
m + n - 1
```

cells.

So if:

```
text
m + n - 1 = odd
```

there is no way to form a valid parentheses string.

Example:

```
text
m = 2
n = 2

len = 2 + 2 - 1
    = 3
```

A 3-character parentheses string can never have equal numbers of (and).

So immediately:

```
C++
return false;
```

20. Why first cell must be (

```
C++
if (grid[0][0] == '(')
    return false;
```

Because every valid parentheses string must start with:

```
text
(
```

If it starts with:

```
text
)
```

then:

```
text
balance = -1
```

immediately.

So there is no point doing DFS.

21. Overall algorithm

The entire algorithm is basically:

```
text
Start at (0,0)
  ↓
Maintain balance
  ↓
'(' → +1
')' → -1
  ↓
If balance < 0 → invalid
  ↓
If balance > remaining → impossible
  ↓
At destination?
  ↓
balance == 0 → valid
  ↓
Otherwise:
  ↓
  / \
  ↓  →
```

```
DFS DFS
 \ /
  OR
  ↓
Memoize result
```

Complexity

There are approximately:

```
text
m × n × (m+n)
```

possible states.

So:

```
text
Time: O(m × n × (m+n))
Space: O(m × n × (m+n))
```

plus recursion stack of $O(m+n)$.

The important thing to remember for interviews is:

`dp[i][j][balance]` means: from cell (i, j) , with this current bracket balance, is it possible to reach the destination with balance 0?

And:

`remaining` tells us how many future cells are available to reduce the current balance. If `balance > remaining`, even making every remaining cell) cannot close all the open brackets, so that state is immediately impossible.

Yes. This is the **bottom-up tabulation version** of the memoization/DFS solution you showed before.

The biggest thing to understand is that the **DP state does not fundamentally change**. What changes is **how we calculate the states**.

1. First remember the memoization version

Your memoization function was:

```
C++
solve(i, j, balance)
```

Meaning:

Starting from cell (i, j) , with the current `balance`, can I reach the destination and finish with balance 0?

Its recurrence was:

```
C++
solve(i, j, balance)
=
  solve(i+1, j, newBalance)
||
  solve(i, j+1, newBalance)
```

where:

```
C++
newBalance = balance + 1 // '('
newBalance = balance - 1 // ')'
```

So memoization was **future** → **future**.

You start at:

```
text
(0, 0)
```

and recursively ask:

```
text
"What can I do from here?"
```

2. Tabulation thinks in the opposite direction

Tabulation changes the question.

Instead of asking:

```
"From here, can I reach the destination?"
```

we ask:

```
"Can I reach this cell with this balance?"
```

Therefore:

```
C++
dp[i][j][balance]
```

now means:

```
Is it possible to reach (i,j) with exactly balance after processing (i,j) ?
```

This is the most important difference.

Memoization

```
text
dp[i][j][balance]
=
Can I finish from here?
```

Tabulation

```
text
dp[i][j][balance]
=
Can I reach here with this balance?
```

Same state dimensions:

```
text
(i, j, balance)
```

but **different meaning/direction**.

3. Why do we initialize this?

```
C++
dp[0][0][1] = true;
```

Look at the starting cell.

Suppose:

```
text
grid[0][0] = '('
```

We start with:

```
text
balance = 0
```

and process the first cell:

```
text
'('
↓
balance = 1
```

Therefore:

```
C++
dp[0][0][1] = true;
```

means:

```
We can reach (0,0) with balance 1.
```

That's our **base case**.

4. Now let's understand the recurrence transformation

This is where memoization → tabulation happens.

In memoization:

```
C++
bool down = solve(i + 1, j, balance, grid);
bool right = solve(i, j + 1, balance, grid);
```

You move:

```
text
current → next
```

So:

```
text
(i,j)
↓
(i+1,j)
```

or:

```
text
(i,j)
→
(i,j+1)
```

In tabulation, we're currently calculating:

```
text
dp[i][j][newBalance]
```

We ask:

"From which previous cell could I have arrived here?"

There are only two possibilities:

```
text
      previous
     /      \
    /        \
   ↑          ←
  /            \
current cell (i,j)
```

Specifically:

From above

```
text
(i-1,j)
```

From left

```
text
(i,j-1)
```

Therefore:

```
C++
if (i > 0 && dp[i-1][j][balance])
```

or:

```
C++
if (j > 0 && dp[i][j-1][balance])
```

5. This is the exact recurrence conversion

Let's write the memoization recurrence first.

Suppose current cell changes the balance:

```
text
newBalance = f(balance, grid[i][j])
```

Memoization:

```

text
dp[i][j][balance]
=
dp[i+1][j][newBalance]
OR
dp[i][j+1][newBalance]

```

Conceptually:

```

text
CURRENT
|
| process current cell
↓
NEW BALANCE
|
|— DOWN
↓
NEXT STATE

OR

CURRENT
|
| process current cell
↓
NEW BALANCE
|
|— RIGHT
↓
NEXT STATE

```

Tabulation reverses that dependency.

Instead of calculating:

```

text
current → next

```

we calculate:

```

text
previous → current

```

So:

```

text
dp[i][j][newBalance]

```

can be reached from:

```

text
dp[i-1][j][balance]

```

or:

```

text
dp[i][j-1][balance]

```

Therefore the recurrence becomes:

```

C++
dp[i][j][newBalance] =
    dp[i-1][j][balance] ||
    dp[i][j-1][balance];

```

That is the **core conversion**.

6. Why is the code looping over `balance`?

Look at:

```

C++
for (int balance = 0; balance <= len; balance++) {

```

We're saying:

```
"Suppose I arrived at the previous cell with balance."
```

Then process the current cell.

```
C++
int newBalance = balance;
```

If current cell is:

```
C++
'('
```

then:

```
C++
newBalance++;
```

If current cell is:

```
C++
')'
```

then:

```
C++
newBalance--;
```

So:

```
text
previous balance
  ↓
process current character
  ↓
new balance
```

7. The most important two lines

This:

```
C++
if (i > 0 && dp[i - 1][j][balance])
    dp[i][j][newBalance] = true;
```

means:

```
If I could reach the cell above with balance, then after processing the current cell, I can reach (i,j) with newBalance.
```

Similarly:

```
C++
if (j > 0 && dp[i][j - 1][balance])
    dp[i][j][newBalance] = true;
```

means:

```
If I could reach the cell on the left with balance, then after processing the current cell, I can reach (i,j) with newBalance.
```

8. Let's visualize it

Suppose current cell is:

```
text
grid[i][j] = ')'
```

So:

```
text
newBalance = balance - 1
```

Imagine:

```

text
dp[i-1][j][2]
  |
  | current = ')'
  ↓
dp[i][j][1]

```

Because:

```

text
2 - 1 = 1

```

Or:

```

text
dp[i][j-1][2]
  |
  | current = ')'
  ↓
dp[i][j][1]

```

Therefore:

```

text
dp[i][j][1] = true

```

if either previous state was reachable.

9. Why don't we explicitly check remaining here?

Your memoization version had:

```

C++
int remaining = (m - 1 - i) + (n - 1 - j);

if (balance > remaining)
    return false;

```

The tabulation version doesn't have it.

Why?

Because remaining was mainly **pruning**.

It helped DFS avoid exploring states that could never succeed.

Tabulation systematically calculates reachable states.

For this problem, the basic tabulation can simply propagate valid balances:

```

text
balance >= 0

```

and finally check:

```

C++
dp[m-1][n-1][0]

```

So:

```

text
Memoization
  ↓
DFS + pruning + memoization

Tabulation
  ↓
Iterate through all states

```

You could add more pruning to the tabulation if desired, but it isn't necessary for the basic recurrence.

10. Why newBalance < 0 is checked

Suppose:

```

text
balance = 0

```

and current cell is:

```
text
```

```
' )'
```

Then:

```
C++
```

```
newBalance = 0 - 1;
```

giving:

```
text
```

```
newBalance = -1
```

That path is invalid.

So:

```
C++
```

```
if (newBalance < 0)
    continue;
```

We don't put negative balances into the DP.

11. Let's do a complete small example

Consider:

```
text
```

```
( (
) )
```

Grid:

```
text
```

```
      j=0   j=1
i=0   (     (
i=1   )     )
```

We start:

```
C++
```

```
dp[0][0][1] = true;
```

because:

```
text
```

```
(
→ balance 1
```

Cell (0,1)

Character:

```
text
```

```
(
```

We consider:

```
text
```

```
balance = 0,1,2,...
```

Suppose:

```
text
```

```
balance = 1
```

Can we reach (0,1) from above?

No:

```
C++
```

```
i > 0
```

is false.

Can we reach from left?

text

```
dp[0][0][1] = true
```

Yes.

Current cell is (:

text

```
newBalance = 1 + 1  
            = 2
```

Therefore:

C++

```
dp[0][1][2] = true;
```

12. Cell (1,0)

Current cell:

text

```
)
```

We have:

text

```
dp[0][0][1] = true
```

Current cell closes one bracket:

text

```
newBalance = 1 - 1  
            = 0
```

Therefore:

C++

```
dp[1][0][0] = true;
```

13. Cell (1,1)

Current:

text

```
)
```

There are two possible previous cells:

text

```
(0,1)  
  |  
  ↓  
(1,1)
```

and:

text

```
(1,0) → (1,1)
```

From (0,1):

text

```
balance = 2
```

Current):

text

```
newBalance = 2 - 1  
            = 1
```

So:

text

```
dp[1][1][1] = true
```

From (1,0):

```
text
balance = 0
```

Current):

```
text
newBalance = -1
```

Invalid.
So the valid state at destination is:

```
text
dp[1][1][1]
```

which means this particular 2x2 grid does **not** have a valid path.
And that's correct because both paths produce:

```
text
( ( )
```

or:

```
text
( ) )
```

Neither is valid.

14. The final answer

At the end:

```
C++
return dp[m - 1][n - 1][0];
```

Why balance 0?
Because a valid parentheses string must have:

```
text
number of '(' = number of ')'
```

Therefore:

```
text
final balance = 0
```

So we're asking:
Can I reach the destination with exactly zero unmatched opening brackets?

If yes:

```
text
true
```

otherwise:

```
text
false
```

15. Memoization vs Tabulation side-by-side

This is the comparison you should remember:

	Memoization	Tabulation
Direction	Future → Future	Previous → Current
Starting point	(0,0,0)	dp[0][0][1] = true
State meaning	Can I finish from here?	Can I reach here?
Movement	Down / Right recursively	Top / Left iteratively
Recurrence	solve(next)	dp[previous] → dp[current]
Calculation	On demand	All reachable states
Stack	Recursion stack	No recursion
Pruning	remaining used	Not required

	Memoization	Tabulation
Final state	Recursive call reaches destination	$dp[m-1][n-1][0]$

16. The mental trick for converting memoization to tabulation

When you have a recursive DP like:

```
C++
solve(state)
```

first identify:

Step 1 — What is the state?

Here:

```
text
(i, j, balance)
```

Step 2 — What does the state mean?

Memoization:

```
text
"Can I finish?"
```

Step 3 — What are the transitions?

```
text
(i, j)
↓
(i+1, j)

(i, j)
→
(i, j+1)
```

Step 4 — Reverse the transitions

For tabulation:

```
text
(i-1, j) → (i, j)

(i, j-1) → (i, j)
```

Step 5 — Find the base state

Starting cell:

```
C++
dp[0][0][1] = true;
```

Step 6 — Iterate in dependency order

Because:

```
text
dp[i][j]
```

depends on:

```
text
dp[i-1][j]
dp[i][j-1]
```

we iterate:

```
C++
for (i = 0 → m-1)
  for (j = 0 → n-1)
```

because by the time we calculate (i, j) , the top and left states have already been calculated.

The core transformation

If you remember only one thing, remember this:

Memoization

```
C++
solve(i,j,balance)
{
    newBalance = process(grid[i][j]);

    return solve(i+1,j,newBalance) ||
           solve(i,j+1,newBalance);
}
```

Tabulation

Reverse the arrows:

```
C++
dp[i][j][newBalance] =
    dp[i-1][j][balance] ||
    dp[i][j-1][balance];
```

So **tabulation** isn't a completely different DP solution. It is the same state-transition graph being evaluated in the opposite direction, without recursion.